Tushe × Noema · Acceptance Worksheet for Friday 16 October

A working document for both teams. Prepared by Noema engineering, 13 October 2026. We'll walk through it together, agree it, sign it, and run the acceptance tests against it, all in one joint session on **Friday 16 October at 18:00 WAT**.

Why this document exists

Our specification says the environment is ready for integration when every acceptance test (AT-01 to AT-20) passes, **or is explicitly accepted as an agreed exception** (spec §25).

Your response shows most of the environment is ready today, and some pieces still need building. This worksheet sets out, test by test, what "ready" means for each one, so that Friday's session can agree it first and then demonstrate it, in one sitting.

How to read the "Friday" column

Every test lands in exactly one of four places:

Outcome	Meaning
Pass	Passes as written in our spec. Shown live on Friday
Pass, amended	Passes under a reworded condition we both sign below. Shown live on Friday
Noema covers	Noema builds this on its own side, so you don't need to. We'll show it to you once it's complete
Tushe, by date	You provide it, on the working-day date in the row

Two rules keep this honest:

1. **Nothing is counted as a pass that isn't one.** An exception stays an exception in the record, with its owner and its date.
2. **A missed date is reported the day it is missed,** by whichever side owns it, with a new date.

---

1. Three definitions we agree first

These three settle four of the twenty tests.

1.1 What "the same material fault sequence" means (AT-01)

The idea in one sentence. If we run the same scenario twice, with the same version and the same seed, **the story of what happened must be identical**: the same faults hit the same things in the same order, every object ends in the same state, and every object took the same path to get there. The clock readings can differ, because two runs never happen at the same moment.

What "the same run" means. Two runs count as "the same run" when all four of these match:

Must match	Example
Scenario key	`commit-then-timeout-01`
Scenario version	`v3`
Definition checksum (SHA-256)	`9f2c…e41a`
Seed	`4271`

Each run happens in its own freshly reset tenant.

What must be identical between the two runs.

#	What	In plain words	How we check it
M1	**The objects**	The scenario creates the same customers, accounts, beneficiaries and payouts, with the same IDs	Compare the ID lists. They must match exactly
M2	**The faults**	The same faults, applied to the same objects, in the same order relative to each other	Compare the ordered list of (fault, target object). It must match exactly
M3	**Each object's path**	For every object, the same sequence of state changes: from which state, to which state, and caused by what (`api` , `provider_webhook` , `watchdog` , `operator` , `backfill`)	Compare each object's ordered list of (from, to, source). It must match exactly
M4	**Where everything ends**	Every object finishes in the same final state, on every lifecycle (`status` , `settlementStatus` , `arrivalStatus`) and in the ledger	Compare the final states and ledger balances. They must match exactly
M5	**The money**	Every amount, fee and balance is the same, to the kobo	Compare the integer kobo values. They must match exactly
M6	**The events**	The same events are emitted, each for the same object, in the same order for that object	Compare each object's ordered list of event types. It must match exactly

What is allowed to differ.

May differ	Why
Wall-clock timestamps (`occurredAt` , `observedAt` , `deliveredAt`)	Two runs never happen at the same moment
How long things took, and the gaps between requests	Network and load vary
Global sequence numbers in the event feed	Other tenants' activity is mixed in
Ordering between **different** objects that have no causal link	Nothing depends on which unrelated object moves first

May differ	Why
Row order where no sort applies	Databases don't promise an order

A worked example (illustrative values; the real paths are whatever your platform produces, as long as both runs produce the same ones). A scenario creates one payout to test account `999904 0000`, "accepted, completed, then reversed by the rail".

	Run A	Run B	Same?
Payout ID	`po_7c1…`	`po_7c1…`	✔ must match
`status` path	pending → processing → completed → reversed	pending → processing → completed → reversed	✔ must match
`arrivalStatus` path	pending → arrived	pending → arrived	✔ must match
Final `settlementStatus`	reversed	reversed	✔ must match
Amount	2,000,000 kobo	2,000,000 kobo	✔ must match
Event order for this payout	created, completed, reversed	created, completed, reversed	✔ must match
`completed` at	18:02:11.204	18:09:43.877	✘ may differ

The test passes when M1 to M6 are identical in both runs. Any difference in M1 to M6 fails it, even one transition or one kobo.

1.2 Where "what actually happened" is read from (AT-02, AT-03)

We accept your counter-proposal in §5.2 instead of a separate privileged-oracle endpoint, on two conditions:

- **A programmatic, read-only partner-administrator credential** that our test harness can use without a browser. It is issued to one named person on our side, and never to the system under test.
- **A refusal test:** an ordinary API key calling any `/partner/...` route is refused.

"What actually happened" is then read from three places:

- the checksummed scenario definition;
- `GET /partner/events`;
- the transition log, once its read endpoint ships (request R2).

1.3 How time is controlled (AT-15)

We accept per-object time instead of an environment-wide clock (§5.1).

Amended AT-15 pass condition: a transition scheduled through the `MMMM` delay fires at its scheduled time, and fires immediately when `POST …/simulate/advance` is called. No global freeze is required.

---

2. The twenty acceptance tests

"Your status" is the Today column from your §8 table.

ID	Test	Your status	Friday	How it is shown, and by whom	Date
AT-01	Deterministic replay	Partial	**Pass, amended** (1.1)	Apply one scenario with the same seed in two freshly reset tenants; compare M1 to M6. Joint	Fri 16 Oct
AT-02	Ground-truth isolation	Pass	**Pass**	Call `/partner/events` with the ordinary API key; it is refused. Tushe	Fri 16 Oct
AT-03	Post-commit timeout	Pass	**Pass, amended** (1.2)	Test account `999902 0000`: the client gets no usable answer; the administrator feed shows the commit. Tushe	Fri 16 Oct
AT-04	Stale state	Fail	**Noema covers**	Noema's proxy returns a prior version after a write, from a fault plan; the authoritative state stays correct. Noema	Shown once complete
AT-05	Event loss	Partial	**Noema covers**, with your delivery log	A configured event is dropped on Noema's side; your delivery log and feed still show it, so the drop is diagnosable. Noema	Shown once complete
AT-06	Duplicate event	Pass	**Pass**	Replay a webhook delivery; the same delivery ID arrives twice, and both attempts are in the log. Tushe	Fri 16 Oct
AT-07	Out-of-order event	Fail	**Noema covers**, with your sequence numbers	Events are reordered on Noema's side; your sequence numbers stay intact, so the reordering is detectable. Noema	Shown once complete
AT-08	Cross-system conflict	Pass	**Pass**	A payout in `completed` with `arrivalStatus: not_arrived`; each lifecycle is read separately and disagrees. Tushe	Fri 16 Oct
AT-09	Ambiguous beneficiary	Fail	**Tushe, by date. Needed for our first demo**	Name enquiry behaves like a real bank: **some lookups return one name, others return several possible matches, or a name that doesn't match the account**, chosen deterministically by the scenario. The client is never silently given one answer. See R9	**Fri 30 Oct**
AT-10	Idempotency conflict	Pass	**Pass**	Same key with a different body returns a deterministic 422 and no second effect. Tushe	Fri 16 Oct
AT-11	Retry after timeout	Pass	**Pass**	Scenario `02`, then a retry with the same key; the business effect is identical across two runs. Tushe	Fri 16 Oct
AT-12	Authorization boundary	Pass	**Pass**	An authenticated but unauthorized principal is refused with a stable business code. Tushe	Fri 16 Oct
AT-13	Policy mutation	Pass	**Pass**	Change a control policy during a scenario; both versions and their effective times can be queried. Tushe	Fri 16 Oct
AT-14	Concurrent update	Partial	**Noema covers**	Our write target in the POC is a case system Noema runs itself, so version tokens and stale-write detection live there. Noema	Shown once complete
AT-15	Time control	Pass, differently	**Pass, amended** (1.3)	Test account `999900 0060`; the transition fires on schedule and also on manual advance. Tushe	Fri 16 Oct
AT-16	Reset & replay	Pass	**Pass**	Apply, reset, apply again; same IDs, no manual database work. Tushe	Fri 16 Oct
AT-17	Ground-truth export	Fail	**Noema covers**, from your read surfaces	Noema's harness assembles the run record from the scenario definition, checksum, administrator feed and transition log. Noema	Shown once complete (needs R2)

ID	Test	Your status	Friday	How it is shown, and by whom	Date
AT-18	Mutation isolation	Pass	**Pass**	The ordinary API key attempts a payout and is refused. Tushe	Fri 16 Oct
AT-19	Audit completeness	Partial	**Tushe, by date**	Actor, request, correlation, state and timestamps on one queryable record (part of E1)	**Fri 23 Oct**
AT-20	Simulator diagnostics	Fail	**Tushe, by date** (interim markers shown Friday)	A diagnostics view that says plainly, for each fault, what was injected, when, against which target, the configured behaviour and the actual outcome, and that tells an injected fault apart from a genuine platform error (E5). Until then, your markers (`SIMULATED_*` reasons, `TRF_SIM_` references, the `source` column) are accepted as an interim. Noema records every fault it injects itself	**Fri 30 Oct** (marker shown Fri 16 Oct)

Friday at a glance:

Outcome	Tests	Count
Pass	AT-02, 06, 08, 10, 11, 12, 13, 16, 18	9
Pass, amended	AT-01, 03, 15	3
Noema covers	AT-04, 05, 07, 14, 17	5
Tushe, by date	AT-09 (30 Oct), AT-19 (23 Oct), AT-20 (30 Oct)	3

3. The twenty-scenario pack

#	Scenario	Your status	How it runs in the POC	Source
1	Normal successful transfer	Yes	`999900 0000`	Tushe
2	Failed before commit	Yes	`999901 0000`	Tushe
3	Commit then timeout	Yes	`999902 0000`	Tushe
4	Debited, not credited	Yes	`completed + arrivalStatus: not_arrived`	Tushe
5	Duplicate webhook	Yes	Replay with the same delivery ID, with webhooks active on our tenants	Tushe
6	Missing webhook	Yes (see note)	The state changes, but the event is withheld on Noema's side	Noema
7	Out-of-order webhook	Partial	Reordered on Noema's side. `999904` (reversal after completion) stays as its own scenario	Noema, plus Tushe for 999904
8	Stale read	No	Noema proxy stale-read profile	Noema
9	Contradictory evidence	Yes	The three lifecycles disagree	Tushe
10	Delayed settlement	Yes	`999900 0060`	Tushe
11	Beneficiary ambiguity	No	Name enquiry returning several or mismatched candidates (R9), plus similar names in seeded histories (R3)	**Tushe, by Fri 30 Oct** (R3 by Fri 23 Oct)
12	Duplicate submission	Yes	Idempotency store	Tushe

#	Scenario	Your status	How it runs in the POC	Source
13	Recovery uncertainty	Yes	The status query returns `unreachable` or `cooling_down`	Tushe
14	Unauthorized principal	Yes	Scopes, module gates, tier caps	Tushe
15	Approval required	Partial	Approval states on payouts: pending, approved, rejected, expired (R10)	**Tushe, by Fri 30 Oct**
16	Policy changed	Yes	Versioned control policies	Tushe
17	Concurrent modification	Partial	On Noema's case system, with version tokens and stale-write detection	Noema
18	Malformed or stale event	No	Injected on Noema's side	Noema
19	Impossible state	Challenged	**Replaced** by a cross-system contradiction (your proposal), for example `999904`	Tushe
20	Composite failure	Partial	Several faults declared in one scenario definition, in a set order (R11), layered with Noema-side transport or event faults	**Tushe, by Fri 30 Oct**, plus Noema

> **Note on scenario 6.** Test account `999905` (never resolves) is a different case from a missing webhook. A missing webhook means the state *changes* but the event never arrives. We keep `999905` as its own case (status-query recovery), and Noema produces the missing-event case on its side.

4. What we need from Tushe, with dates

These are essential to our build and to our first demo. Your indicative dates in §11 (24 Oct to 12 Dec) would block us, so **we're asking for everything still on our list to land within October**, in this order.

#	Request	Why it matters to us	Date
R1	**Tenants and credentials, in two batches.** Batch 1: **3 to 5 test-mode tenants**, a read-only API key for each, one programmatic read-only administrator credential, and one test-mode credential that can create payouts for building scenario histories (held by one named Noema person, never given to the system under test). Batch 2: the remaining tenants, to about **10** in total	Our team starts integrating on day one; parallel runs need one tenant each	**Batch 1: Thu 15 Oct. Batch 2: Tue 20 Oct**

#	Request	Why it matters to us	Date
R2	**The read surfaces (E1):** the transition-history endpoint (STM-03), transaction history (REC-01), the correlated timeline (REC-02), causation exposed on events (EVT-01, AUD-04), audit query by transaction, account, customer and correlation ID (AUD-05), and actor on the transition record (AUD-02, AT-19)	This is how we, and you, establish what actually happened	**No later than Fri 23 Oct**
R3	**Seeded transaction histories:** scenario definitions that seed **past payouts**, each with a set beneficiary name, amount, outcome and `occurredAt` date, including several with similar names and the same amount on the same day	Our core cases start from a customer's history: "which of these three transfers did you mean?"	**No later than Fri 23 Oct**
R4	**Switch state as its own queryable dimension:** the raw code, a normalised status, and whether it is final (your additive change under SE-01)	Our spec lists the switch as an independent system (SE-01, P0); our conflict cases depend on it	**No later than Fri 23 Oct**
R5	No behaviour change on the sandbox from 13 Oct to 6 Nov without 48 hours' notice, and the sandbox version on every response	Replay depends on a stable environment	**From now**
R6	Written confirmation that the status-query endpoint can never move money, reverse a debit or change balances	We classify it as a read	**Fri 16 Oct**
R7	Test-mode rate limits and quotas, and the sandbox's hosting region	We run large repeated suites, and we have a latency budget	**Fri 16 Oct**
R8	Answers to the questions in section 6	So nothing surprises us later	**Fri 16 Oct**
R9	**Ambiguous name enquiry (E6, AT-09).** Name enquiry behaves like a real bank: some lookups return a single exact name; some return **several possible matches**; some return a name that **doesn't match** the account. Which one happens is set by the scenario and repeats on replay	Resolving the right recipient is at the heart of what we're building, **and it's in our first demo**	**Fri 30 Oct**

#	Request	Why it matters to us	Date
R10	**Payout approval states (E6, AUTH-02):** pending, approved, rejected and expired approvals on payouts, settable by scenario	Real fintech flows wait on approvals; we must reason about them correctly	**Fri 30 Oct**
R11	**Scenario-definition richness (the rest of E4):** adversarial fixtures (duplicate customers, similar names, multiple matching accounts, repeated amounts, stale records, conflicting metadata), faults and event schedules declared inside the definition, ordered composition of several faults, and medium and large data tiers	A trivial single-record lookup must never be enough to get the right answer (DATA-04)	**Fri 30 Oct**
R12	**The diagnostics view (E5, AT-20, OBS-01):** for every fault, what was injected, when, against which target, the configured behaviour and the actual outcome, plus a clear injected-versus-genuine-error flag	When a run fails, we must be able to tell in seconds whether it was our bug or the simulator	**Fri 30 Oct**
R13	**Webhooks kept active on our tenants,** delivering to a public HTTPS endpoint we register, alongside the event feed	We read the feed as our main source, and need live webhooks to test replay and deduplication	**With batch 1 (Thu 15 Oct)**

5. What Noema builds on its own side

So your effort goes where only you can help, these are built by us. You don't need to build them for our POC.

- **Event-delivery faults (E3):** drop, delay, reorder and malformed delivery, injected on our side.
- **The stale-read profile:** our proxy returns old versions on purpose.
- **Version tokens and stale-mutation detection on our write path.** Our writes go to a case system we run, so these live there.
- **Run export (E7):** we assemble run records from your read surfaces (R2).
- **A case or dispute domain:** out of scope for you.
- **A privileged oracle endpoint (GT-02):** replaced by your administrator surface, on the conditions in 1.2.
- **Transport faults (timeouts, resets, gateway errors):** our own proxy; nothing to buy and nothing in your money path.

6. Questions we need answered by Friday

1. **Requirements not in your response.** Part Two has no rows for our ledger requirements (LED-01 to LED-06, §4.3) and answers the six idempotency requirements (IDP-01 to IDP-06, §11) in one combined row. Please add rows for all twelve. Three matter most to us:
   - LED-04: one leg posted and another delayed or uncertain;
   - LED-05: money held in integer kobo;
   - IDP-06: how long idempotency keys are kept.
2. **Reset.** Does reset also remove payouts created through the API on a scenario's accounts, or only the objects the scenario seeded?
3. **IDs.** Do payouts created through the API in a freshly applied scenario get the same IDs every time if we send the same idempotency key and reference?
4. **Tenants.** We expect to need about ten test tenants across developers, CI, nightly runs and demos. Can tenants be created through an API, or only by your team?
5. **Feed and webhooks.** We'll read `GET /v1/events` by sequence cursor as our main source, and we'd like **webhooks kept active** on our tenants so we can test replay and deduplication. What do you need from us to register our public HTTPS endpoint?
6. **Integer kobo, everywhere (LED-05).** Please confirm that **every amount on every endpoint and every event is an integer number of kobo**: payouts, fees, balances, ledger entries and webhook payloads. No floats, no decimal naira strings. This matters a great deal to us, because floating-point rounding on money is exactly the kind of silent error our product exists to prevent.
7. `deliveredAt` **(TIM-03).** Please confirm `deliveredAt` is now available, and update the TIM-03 row (see our response, section 4, item 6).
8. **Sandbox availability.** What availability should we expect between 13 October and 6 November, and who do we contact when it's down?

7. Friday's session, step by step (18:00–20:30 WAT)

Time	Step
18:00	**Walk through the worksheet together.** Agree the three definitions (section 1), every test and scenario row (sections 2 and 3), and the dates in section 4. Note any amendment in the document
18:45	**Sign the agreed worksheet.** Then confirm tenant, keys and sandbox version, and record the version
18:55	**AT-02 and AT-18** (credential boundaries)
19:05	**AT-16, then AT-01** (reset, then deterministic replay against M1 to M6)
19:30	**AT-03, AT-11, AT-10** (commit-then-timeout, retry, idempotency)
19:50	**AT-08, AT-06, AT-15** (contradiction, duplicate delivery, time)
20:05	**AT-12, AT-13** (authorization, policy versions)
20:15	**AT-20 interim** (your simulated-fault markers)

Time	Step
20:20	Read back the record: every test and its outcome, every exception with its owner and date
20:30	Both sides sign the record

Noema will show you AT-04, AT-05, AT-07, AT-14 and AT-17 once each is complete, and share the evidence with you at the same time.

8. Sign-off

By signing, both teams agree the definitions in section 1, the outcome of every row in sections 2 and 3, and the dates in section 4.

	Name	Role	Date
Tushe			
Noema			